Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
Университетский технологический колледж
Кафедра Информационные системы в экономике

Отчет защищен с оценкой _____

Преподаватель С.В.Умбетов

«___» _____ 2026 г.

Отчет по лабораторной работе №4
по дисциплине «Инженерно-техническая поддержка сопровождения интеллектуальных информационных систем»
«Контейнеризация компонентов ИИС и управление зависимостями (Docker)»

ЛР 09.02.08.19.004

Студент группы 1ИИС-41 Н.Ю. Потапов
группа инициалы., фамилия
Преподаватель старший преподаватель, доцент С.В. Умбетов
должность, ученая степень инициалы., фамилия

Барнаул 2026

Цель работы: Освоить принципы контейнеризации интеллектуальных информационных систем (ИИС) с использованием Docker. Изучить создание Dockerfile для приложений с тяжёлыми зависимостями, методы оптимизации Docker-образов (multi-stage build), описание многоконтейнерных приложений через docker-compose.yml и организацию персистентного хранения данных через volumes.

Задачи к работе:

1. Установить Docker и Docker Compose, проверить корректность установки.
2. Создать структуру каталога проекта и подготовить файл зависимостей (requirements.txt).
3. Написать multi-stage Dockerfile на базе python:3.11-slim и создать файл .dockerignore.
4. Подготовить исходный код приложения (модуль модели и HTTP-API).
5. Описать docker-compose.yml с сервисами api и db (PostgreSQL) с именованными volumes.
6. Выполнить сборку образов и зафиксировать размер итогового образа.
7. Запустить систему и проверить работоспособность API.
8. Убедиться в сохранности данных в volumes при перезапуске контейнеров.
9. Добавить сервис Redis для кэширования предсказаний.

Задание принял:  Потапов Н.Ю.

Подпись ФИО

Ход работы:

Задание 1. Установить Docker и проверить версии.

Ход выполнения: Выполнил установку Docker Engine и плагина Docker Compose в систему. Добавил текущего пользователя в группу docker для запуска без sudo. Проверил версии установленных компонентов с помощью команд docker --version и docker compose version.

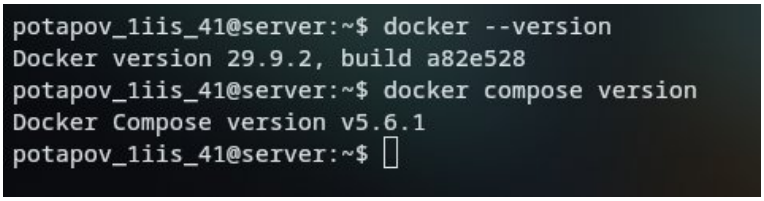


Рисунок 1. Проверка версий Docker и Docker Compose

Задание 2. Создать структуру проекта и файл зависимостей.

Ход выполнения: Создал корневой каталог iis-docker и подкаталог app. Внутри app создал файл requirements.txt, в котором явно зафиксировал версии необходимых пакетов: fastapi, uvicorn, scikit-learn, numpy, pandas и psycopg2-binary.

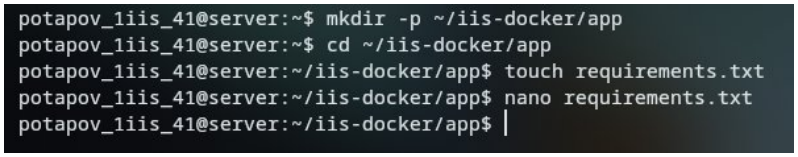


Рисунок 3. Создание и редактирование файла requirements.txt

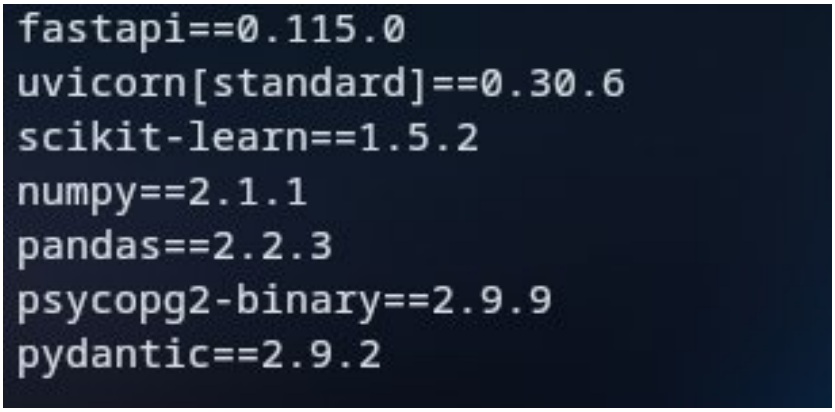


Рисунок 4. Содержание файла requirements.txt

Задание 3. Создать файл .dockerignore.

Ход выполнения: Создал файл .dockerignore для исключения лишних файлов (кэш, .git, .env) из контекста сборки.

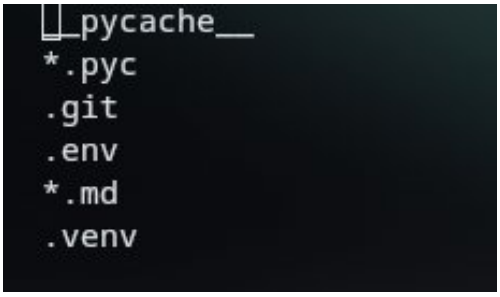


Рисунок 6. Содержание файла .dockerignore

Задание 4. Написать multi-stage Dockerfile

Ход выполнения: Создал файл Dockerfile, использующий технику multi-stage build. На стадии builder установил компиляторы и собрал виртуальное окружение. На стадии runtime скопировал только готовое окружение и код в легкий образ python:3.11-slim, а также создал непривилегированного пользователя appuser для безопасности.

```
FROM python:3.11-slim AS builder

WORKDIR /build

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    gcc \
    && rm -rf /var/lib/apt/lists/*

COPY app/requirements.txt .

RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

FROM python:3.11-slim AS runtime

WORKDIR /app

COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

COPY app/ .

RUN useradd -m -u 1000 appuser && \
    chown -R appuser:appuser /app
USER appuser

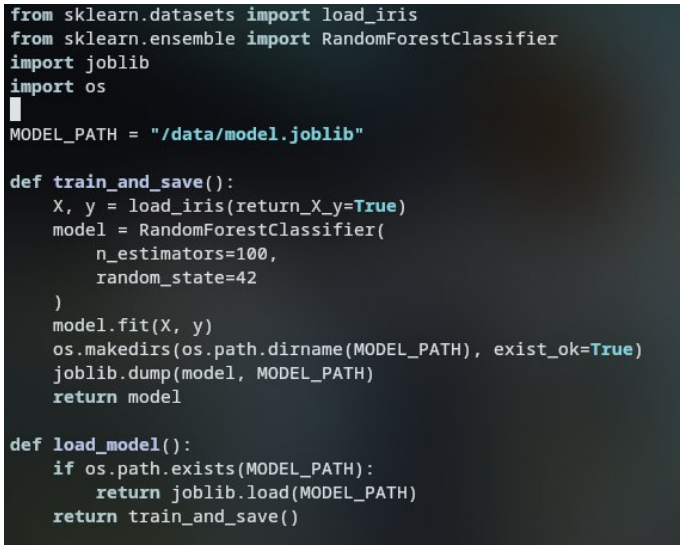
EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Рисунок 5. Содержание файла Dockerfile

Задание 5. Подготовить исходный код приложения.

Ход выполнения: В каталоге app создал модуль model.py, который обучает модель RandomForest на наборе данных Iris и сохраняет/загружает её из каталога /data. Создал файл main.py с HTTP-API на FastAPI, реализующим эндпоинт проверки состояния (/) и эндпоинт предсказания класса (/predict).



```python
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
import joblib
import os

MODEL_PATH = "/data/model.joblib"

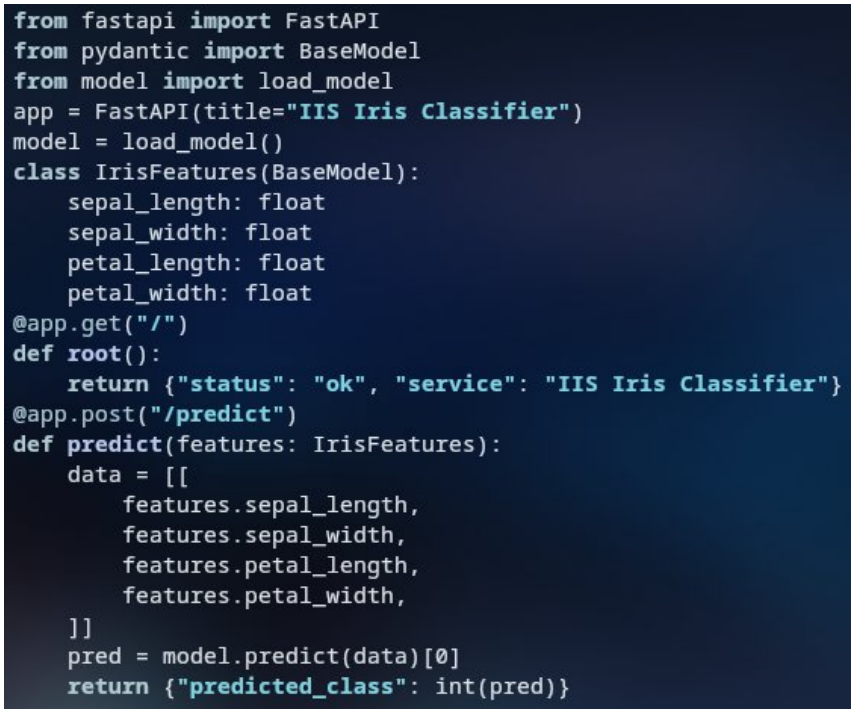
def train_and_save():
    X, y = load_iris(return_X_y=True)
    model = RandomForestClassifier(
        n_estimators=100,
        random_state=42
    )
    model.fit(X, y)
    os.makedirs(os.path.dirname(MODEL_PATH), exist_ok=True)
    joblib.dump(model, MODEL_PATH)
    return model

def load_model():
    if os.path.exists(MODEL_PATH):
        return joblib.load(MODEL_PATH)
    return train_and_save()
```

Рисунок 6. Исходный код модели (model.py)

Задание 6. Подготовить HTTP-API.

Ход выполнения: Создал файл main.py с HTTP-API на FastAPI, реализующим эндпоинт проверки состояния (/) и эндпоинт предсказания класса (/predict) по четырем признакам цветка.



```python
from fastapi import FastAPI
from pydantic import BaseModel
from model import load_model
app = FastAPI(title="IIS Iris Classifier")
model = load_model()
class IrisFeatures(BaseModel):
    sepal_length: float
    sepal_width: float
    petal_length: float
    petal_width: float
@app.get("/")
def root():
    return {"status": "ok", "service": "IIS Iris Classifier"}
@app.post("/predict")
def predict(features: IrisFeatures):
    data = [[
        features.sepal_length,
        features.sepal_width,
        features.petal_length,
        features.petal_width,
    ]]
    pred = model.predict(data)[0]
    return {"predicted_class": int(pred)}
```

Рисунок 7. Исходный код HTTP-API (main.py)

Задание 7. Описать docker-compose.yml.

Ход выполнения: Создал файл docker-compose.yml, в котором описал два сервиса: api, собираемый из локального Dockerfile, и db, использующий официальный образ postgres:16-alpine. Настроил именованные volumes model_data и postgres_data для персистентного хранения данных.

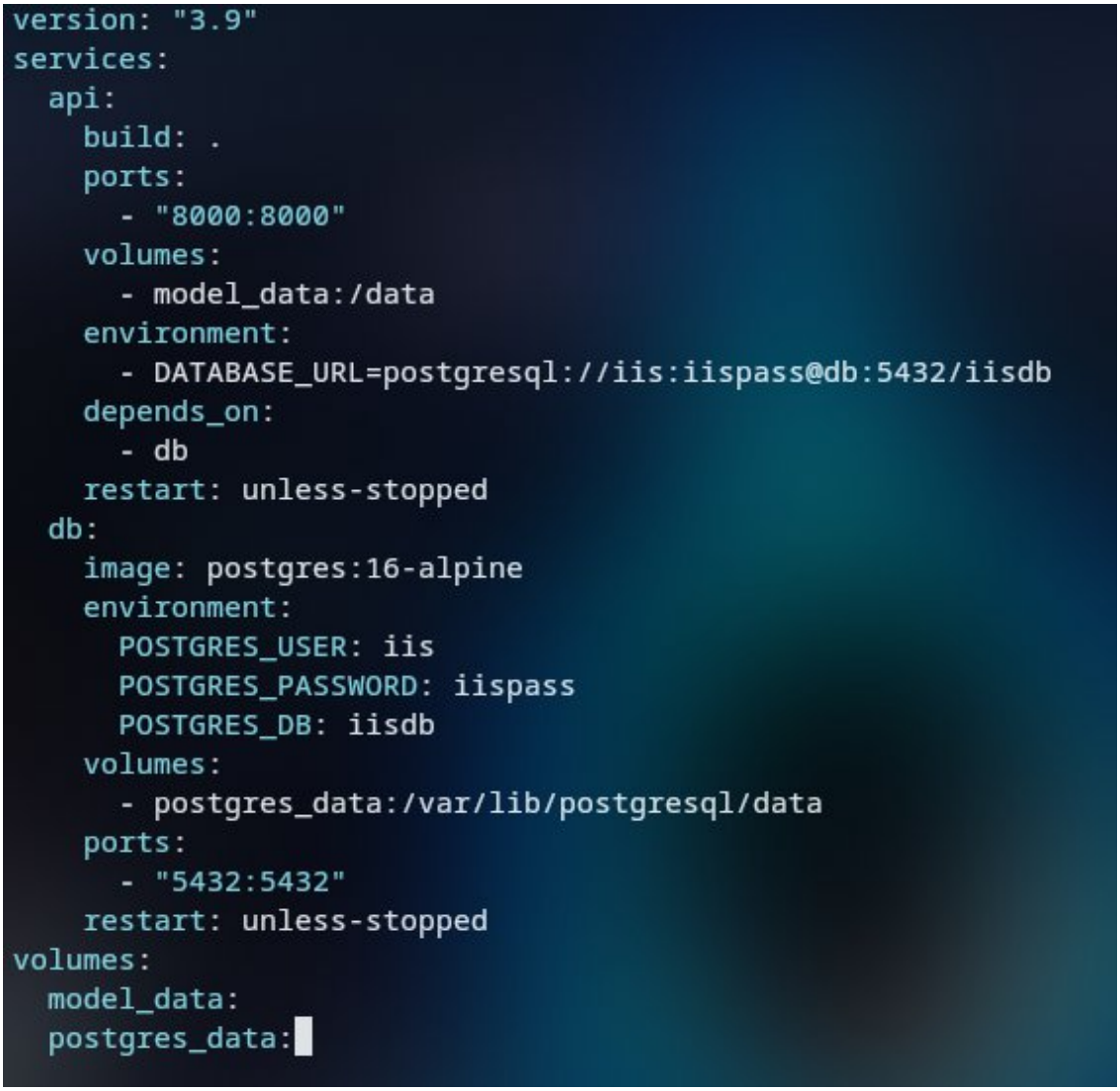


Рисунок 8. Содержание файла docker-compose.yml

Задание 8. Выполнить сборку образов.

Ход выполнения: Из корневого каталога проекта выполнил команду docker compose build для сборки образов. После успешной сборки проверил размер итогового образа с помощью команды docker images и зафиксировал результат.

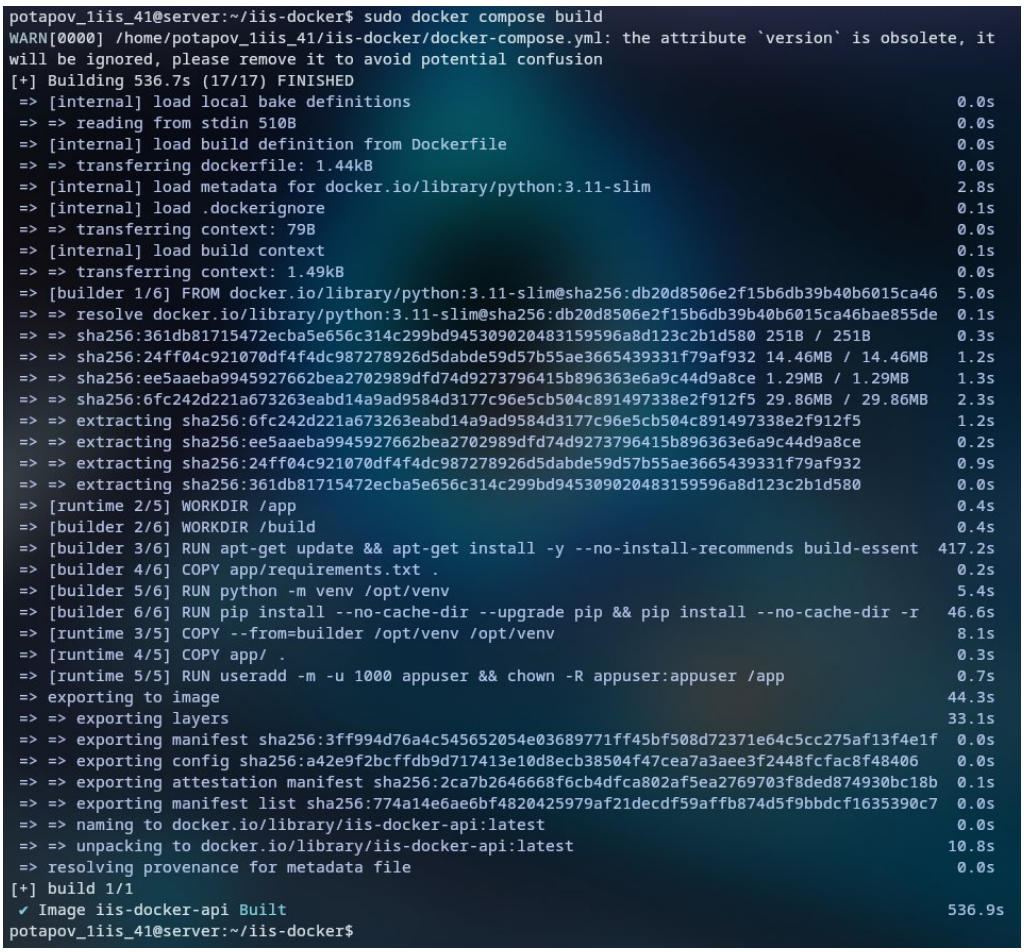


Рисунок 9. Процесс сборки образов

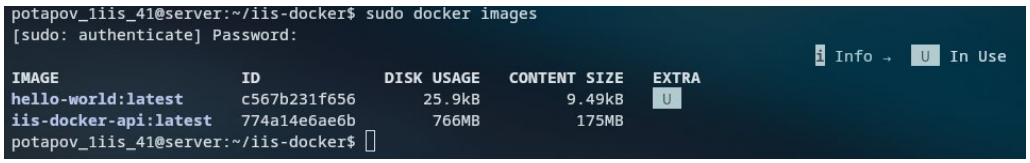


Рисунок 10. Размер итогового Docker-образа

Задание 9. Запустить систему и проверить состояние.

Ход выполнения: Запустил многоконтейнерное приложение в фоновом режиме командой docker compose up -d. Проверил состояние всех запущенных сервисов с помощью команды docker compose ps.

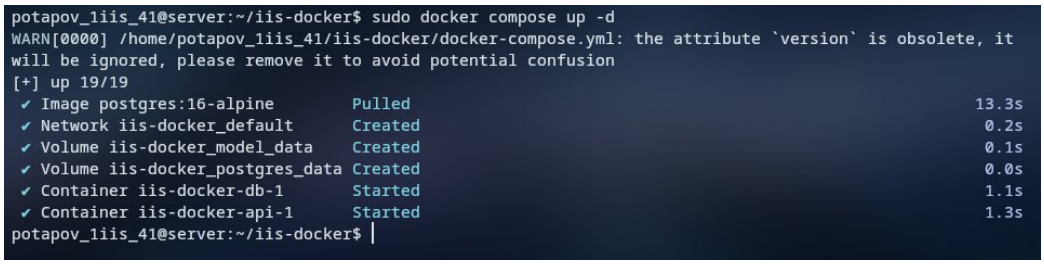


Рисунок 11. Запуск контейнеров



Рисунок 12. Статус запущенных сервисов

Задание 10. Проверить работоспособность API.

Ход выполнения: Проверил корневой эндпоинт сервиса с помощью curl. Затем отправил POST-запрос на эндпоинт /predict с тестовыми признаками цветка ириса и получил ответ с предсказанным классом.



Рисунок 13. Проверка корневого эндпоинта

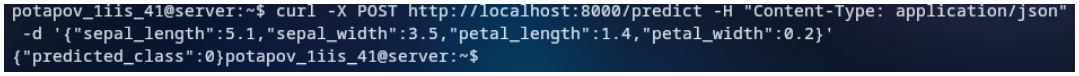


Рисунок 14. Запрос предсказания и ответ API

Задание 11. Проверить содержимое volume.

Ход выполнения: Выполнил команду docker volume ls для просмотра списка volumes проекта. С помощью docker volume inspect получил подробную информацию о volume с моделью. Зашел внутрь контейнера api командой docker compose exec api ls -la /data и убедился, что файл модели сохранен.

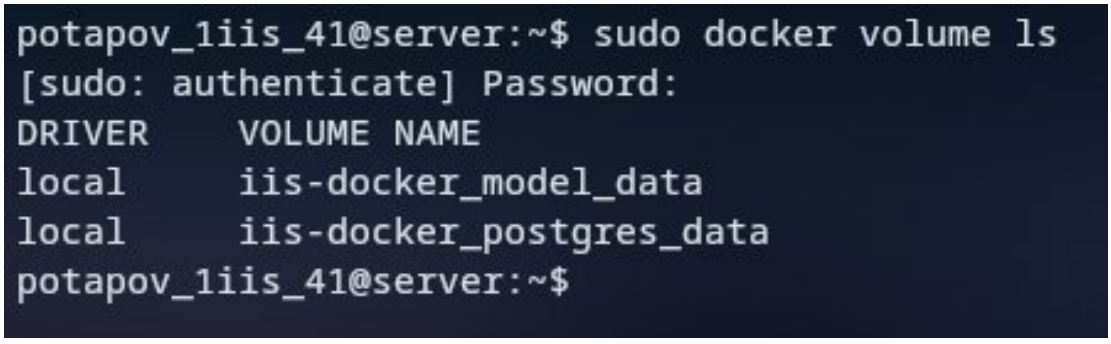


Рисунок 15. Список volumes проекта

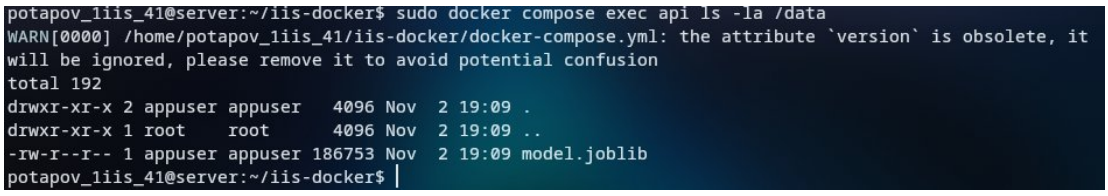


Рисунок 16. Содержимое директории /data внутри контейнера

Задание 12. Перезапустить систему и убедиться в сохранности данных.

Ход выполнения: Остановил и удалил контейнеры командой docker compose down, при этом volumes сохранились. Повторно запустил систему командой docker compose up -d. Снова проверил эндпоинт /predict и убедился, что модель успешно загрузилась из volume и предсказания работают корректно.

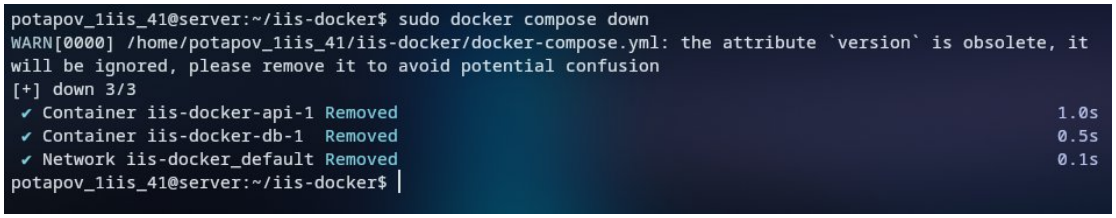


Рисунок 17. Остановка контейнеров

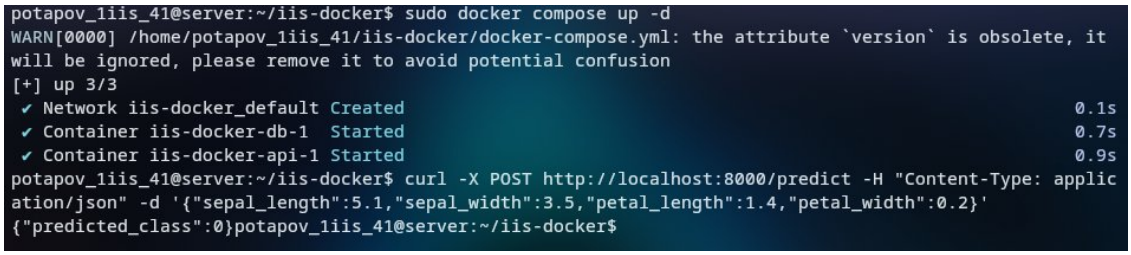
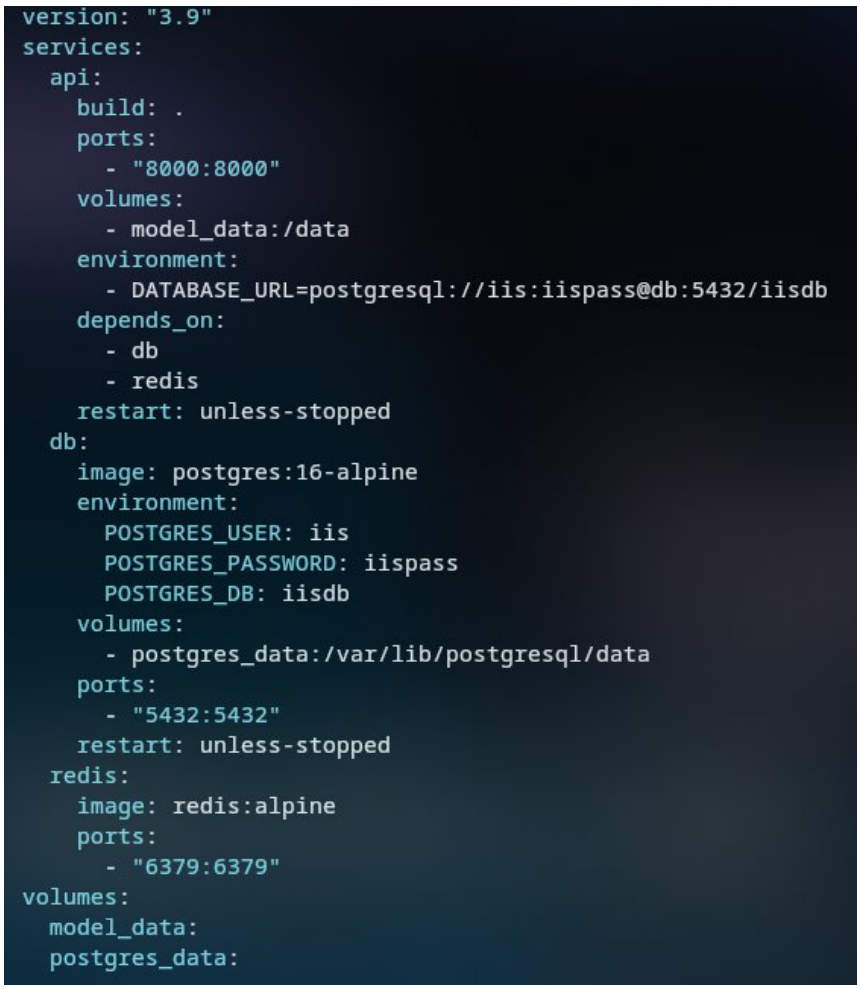


Рисунок 18. Повторный запуск и проверка API

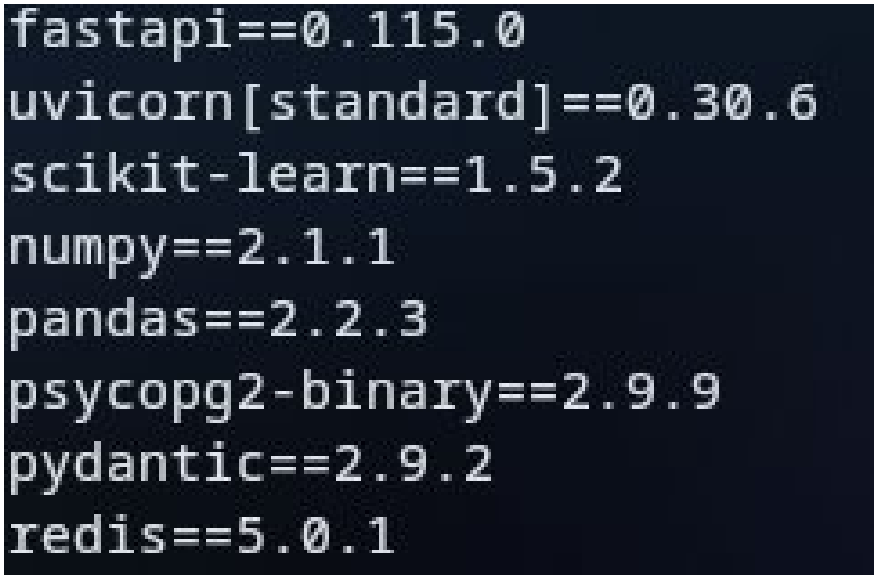
Задание 12. Добавить сервис Redis для кэширования предсказаний.

Ход выполнения: Для оптимизации работы системы добавил сервис кэширования Redis. В файл docker-compose.yml добавил новый сервис redis на базе образа redis:alpine с публикацией порта 6379. Обновил файл app/requirements.txt, добавив зависимость redis==5.0.1. Модифицировал файл app/main.py: добавил инициализацию клиента Redis, функцию хеширования входных данных для создания уникального ключа кэша, логику проверки наличия готового предсказания в кэше перед обращением к модели, а также сохранение результата в кэш с временем жизни (TTL) 300 секунд. Пересобрал образы командой docker compose build --no-cache и перезапустил систему командой docker compose up -d. Проверил работу кэширования, отправив два одинаковых POST-запроса на эндпоинт /predict. Убедился, что при втором запросе предсказание возвращается мгновенно из кэша Redis, не нагружая модель машинного обучения.



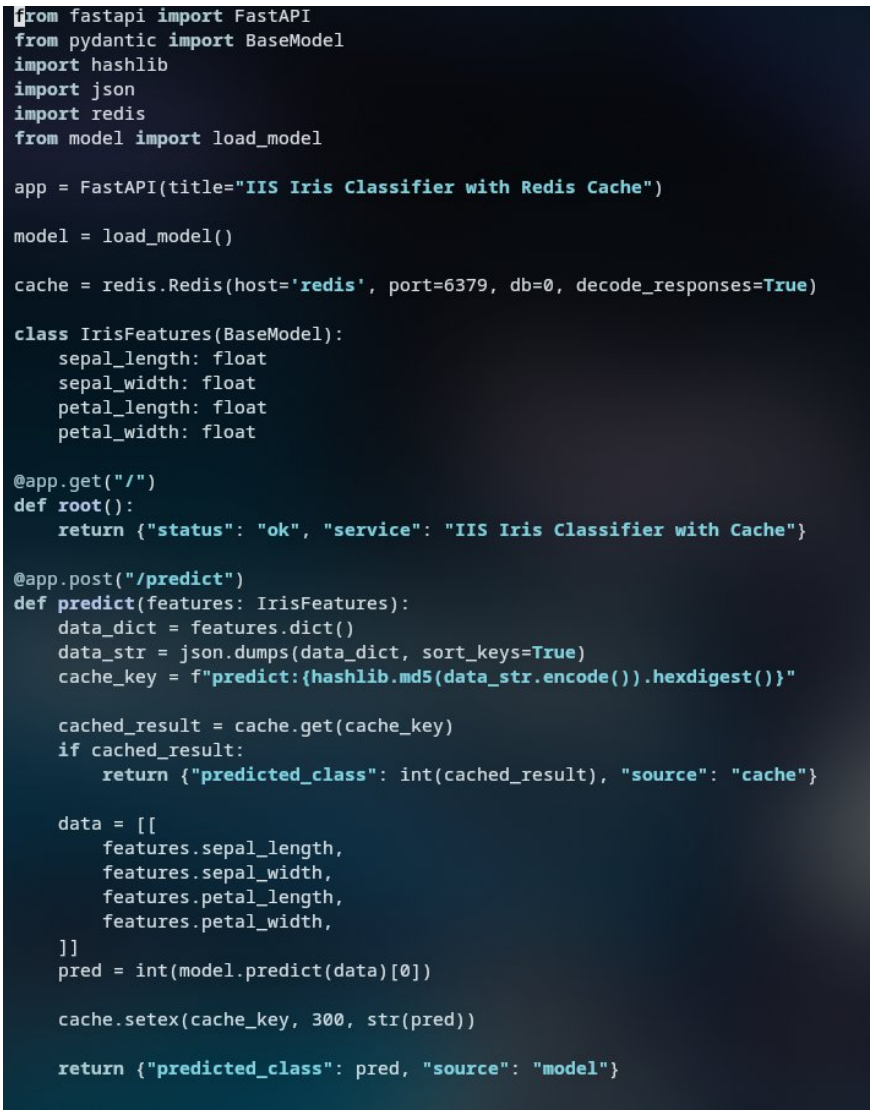
```yaml
version: "3.9"
services:
  api:
    build: .
    ports:
      - "8000:8000"
    volumes:
      - model_data:/data
    environment:
      - DATABASE_URL=postgresql://iis:iispass@db:5432/iisdb
    depends_on:
      - db
      - redis
    restart: unless-stopped
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: iis
      POSTGRES_PASSWORD: iispass
      POSTGRES_DB: iisdb
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    restart: unless-stopped
  redis:
    image: redis:alpine
    ports:
      - "6379:6379"
volumes:
  model_data:
  postgres_data:
```

Рисунок 19. Добавление сервиса Redis в docker-compose.yml



```
fastapi==0.115.0
uvicorn[standard]==0.30.6
scikit-learn==1.5.2
numpy==2.1.1
pandas==2.2.3
psycopg2-binary==2.9.9
pydantic==2.9.2
redis==5.0.1
```

Рисунок 20. Обновленный файл requirements.txt с пакетом redis



```python
from fastapi import FastAPI
from pydantic import BaseModel
import hashlib
import json
import redis
from model import load_model

app = FastAPI(title="IIS Iris Classifier with Redis Cache")

model = load_model()

cache = redis.Redis(host='redis', port=6379, db=0, decode_responses=True)

class IrisFeatures(BaseModel):
    sepal_length: float
    sepal_width: float
    petal_length: float
    petal_width: float

@app.get("/")
def root():
    return {"status": "ok", "service": "IIS Iris Classifier with Cache"}

@app.post("/predict")
def predict(features: IrisFeatures):
    data_dict = features.dict()
    data_str = json.dumps(data_dict, sort_keys=True)
    cache_key = f"predict:{hashlib.md5(data_str.encode()).hexdigest()}"

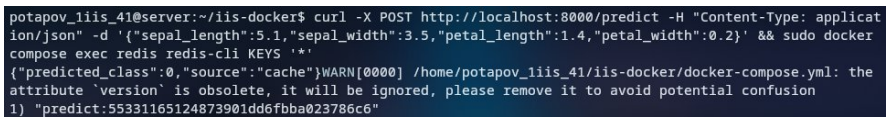
    cached_result = cache.get(cache_key)
    if cached_result:
        return {"predicted_class": int(cached_result), "source": "cache"}

    data = [[
        features.sepal_length,
        features.sepal_width,
        features.petal_length,
        features.petal_width,
    ]]
    pred = int(model.predict(data)[0])

    cache.setex(cache_key, 300, str(pred))

    return {"predicted_class": pred, "source": "model"}
```

Рисунок 21. Модификация кода main.py для проверки и сохранения кэша



```
potapov_1iis_41@server:~/iis-docker$ curl -X POST http://localhost:8000/predict -H "Content-Type: applicat
ion/json" -d '{"sepal_length":5.1,"sepal_width":3.5,"petal_length":1.4,"petal_width":0.2}' && sudo docker
compose exec redis redis-cli KEYS '*'
{"predicted_class":0,"source":"cache"}WARN[0000] /home/potapov_1iis_41/iis-docker/docker-compose.yml: the
attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
1) "predict:55331165124873901dd6fbba023786c6"
```

Рисунок 22. Проверка работы кэширования через повторный запрос и просмотр ключей в redis-cli

Вывод:

В ходе лабораторной работы я освоил контейнеризацию ИИС с помощью Docker. На практике разработал multi-stage Dockerfile для оптимизации размера образа, настроил взаимодействие сервисов (FastAPI, PostgreSQL, Redis) через docker-compose и организовал персистентное хранение данных с помощью volumes. В рамках дополнительного задания внедрил кэширование предсказаний на Redis. Полученные навыки позволяют эффективно упаковывать, масштабировать и развертывать ИИС в изолированной среде.

Ссылка на мой репозиторий с работами

Ответы на контрольные вопросы:

1. Что такое Docker-образ и чем он отличается от контейнера?

Docker-образ — это неизменяемый шаблон, содержащий файловую систему и метаданные для запуска приложения. Контейнер — это запущенный экземпляр образа, имеющий собственное изолированное пространство процессов и сетевой стек.

2. Для чего используется multi-stage build и какую проблему он решает при контейнеризации систем машинного обучения?

Multi-stage build позволяет использовать несколько стадий в одном Dockerfile. Это решает проблему большого размера образов: на первой стадии устанавливаются компиляторы и собираются зависимости, а в финальный легковесный образ копируются только готовые артефакты, что значительно уменьшает итоговый размер контейнера.

3. Почему рекомендуется использовать образы python:*-slim или alpine вместо полноразмерных?

Образы slim и alpine содержат только минимально необходимый набор системных библиотек, что существенно уменьшает размер итогового образа, сокращает время сборки и снижает поверхность для потенциальных уязвимостей безопасности.

4. Что такое слой (layer) в Docker и почему важен порядок инструкций в Dockerfile?

Слой — это результат выполнения одной инструкции Dockerfile. Порядок инструкций важен для эффективного кэширования: если изменяются только нижние слои, верхние слои берутся из кэша, что ускоряет пересборку образа.

5. Для чего предназначен файл .dockerignore?

Файл .dockerignore исключает ненужные файлы и директории (например, .git, pycache, .env) из контекста сборки, что уменьшает размер передаваемых данных и ускоряет процесс сборки образа.

6. В чём различие между bind-mount и named volume?

Bind-mount монтирует конкретный путь из хостовой системы в контейнер и зависит от структуры файловой системы хоста. Named volume управляется

самим Docker, хранится в специальном каталоге и обеспечивает персистентность данных независимо от жизненного цикла контейнера.

7. Как обеспечить сохранение состояния модели и базы данных при пересоздании контейнеров?

Необходимо использовать именованные volumes (named volumes) в docker-compose.yml, монтируя их в директории, где приложение сохраняет файлы модели и данные СУБД. При удалении и пересоздании контейнеров данные в volumes остаются нетронутыми.

8. Что выполняет инструкция COPY --from=builder?

Она копирует файлы или директории из ранее собранной стадии (в данном случае builder) в текущую стадию (runtime), позволяя перенести только необходимые артефакты, такие как виртуальное окружение, без компиляторов и временных файлов.

9. Зачем запускать процесс приложения от имени непривилегированного пользователя?

Запуск от непривилегированного пользователя повышает безопасность контейнера. В случае компрометации приложения злоумышленник не получит прав root внутри контейнера, что ограничивает возможности для атаки на хостовую систему.

10. Как просмотреть логи отдельного сервиса в docker-compose?

Логи конкретного сервиса можно просмотреть с помощью команды docker compose logs -f <имя_сервиса>, например, docker compose logs -f api.

11. Какими способами можно уменьшить размер образа при использовании PyTorch или TensorFlow?

Для уменьшения размера образа можно использовать multi-stage build, базовые образы с пометкой slim, устанавливать только CPU-версии библиотек (если CUDA не требуется), очищать кэш пакетного менеджера (apt clean, pip install --no-cache-dir) и объединять команды RUN в одну инструкцию.

12. Что означает параметр depends_on и гарантирует ли он готовность зависимого сервиса?

Параметр depends_on задает порядок запуска контейнеров, но не гарантирует, что сервис полностью готов к приему соединений (например, база данных может еще инициализироваться). Для ожидания готовности используются healthcheck и условия starts_depend.

13. Как определить, какие volumes использует проект?

Список volumes можно посмотреть командой docker volume ls, а подробную информацию о конкретном volume проекта — командой docker volume inspect <имя_volume>.

14. В чём преимущество контейнеризации ИИС по сравнению с виртуальными машинами?

Контейнеры используют ядро хостовой ОС, что обеспечивает гораздо более быстрый запуск, меньшее потребление ресурсов (памяти и процессора) и компактный размер образов по сравнению с полноценными виртуальными машинами, которым требуется своя гостевая ОС.

15. Какие риски возникают при хранении паролей и секретов непосредственно в docker-compose.yml?

Хранение паролей в docker-compose.yml приводит к их попаданию в систему контроля версий (например, Git), что создает риск утечки учетных данных. Для безопасного хранения следует использовать Docker secrets, переменные окружения из файлов .env или специализированные vault-системы.